BÁO CÁO CHI TIẾT DỰ ÁN DIACARE-MAS

Hệ đa tác tử hỗ trợ tư vấn phòng ngừa tiểu đường: thiết kế, cài đặt, triển khai và đánh giá

Lý Hải Quân - B23DCCN676 - Nhóm lớp 05

Học viện Công nghệ Bưu chính Viễn thông - Tháng 11/2026

Mục lục

1. Tổng quan 3
1.1. Dự án là gì 3
1.2. Xuất phát điểm 3
1.3. Mục tiêu và phạm vi 4
2. Kiến trúc 5
2.1. Sơ đồ tổng thể 5
2.2. Vì sao là pipeline cố định chứ không phải vòng lặp agent 5
2.3. Luồng xử lý một câu hỏi 6
3. Thiết kế chi tiết từng thành phần 6
3.1. Trạng thái dùng chung 6
3.2. Orchestrator 7
3.3. Memory: bộ nhớ ngắn hạn và dài hạn 7
3.4. Lab: đọc và xếp mức xét nghiệm 8
3.5. Predictor: dự đoán nguy cơ 8
3.6. Retriever: tìm tài liệu 9
3.7. Graph: đồ thị tri thức 9
3.8. Nutrition: dinh dưỡng món ăn 13
3.9. RL: học chọn trọng tâm tư vấn từ phản hồi 13
3.10. Context: ngân sách token 15
3.11. Advisor và mô hình ngôn ngữ 16
3.12. Rules: rào chắn an toàn 16
4. Cơ sở dữ liệu 17
5. Giao diện 18
5.1. Thanh bên 18
5.2. Trang Tư vấn 19
5.3. Trang Hoạt động agent 21
5.4. Trang Lịch sử và Thống kê 22

6. Triển khai 22
6.1. Chạy trên máy cá nhân 22
6.2. Chạy trên Kaggle 22
7. Kiểm thử và đánh giá 23
7.1. Unit test 23
7.2. Benchmark hội thoại 23
7.3. Kết quả ablation harness 24
7.4. Khi đổi model 24
8. Nhật ký lỗi và bài học 24
9. Hạn chế và hướng phát triển 26
Tài liệu tham khảo 27

1. Tổng quan

1.1. Dự án là gì

DiaCare-MAS là một trợ lý trò chuyện bằng tiếng Việt về phòng ngừa và chăm sóc bệnh tiểu đường. Người dùng tạo hồ sơ sức khỏe, có thể nhập kết quả xét nghiệm, rồi đặt câu hỏi như "tôi nên ăn gì", "đi tiểu nước có kiến bâu có sao không", "HbA1c 6,8% là cao hay thấp". Hệ trả lời bằng một mô hình ngôn ngữ lớn chạy cục bộ, có trích nguồn tài liệu, và luôn kèm lời nhắc tham khảo bác sĩ.

Bên trong, câu trả lời không do một mô hình duy nhất tạo ra. Mười tác tử (agent) chia nhau công việc: một agent nhớ thông tin bệnh nhân, một agent đọc xét nghiệm, một mạng nơ-ron dự đoán nguy cơ, một agent tìm tài liệu, một agent tra đồ thị tri thức, một agent tra dinh dưỡng, một agent học tăng cường chọn trọng tâm tư vấn và học từ phản hồi của người dùng, một agent cắt ngữ cảnh, mô hình ngôn ngữ viết bản nháp, và một agent luật kiểm tra trước khi trả lời. Một Orchestrator điều phối cả mười.

Dự án làm cho mục đích học tập. **Nó không thay thế bác sĩ** và không được dùng để chẩn đoán.

- Mã nguồn: https://github.com/lyhaiquan/diabet-agent
- Ngôn ngữ: Python 3.14. Giao diện Streamlit, mô hình ngôn ngữ qua Ollama, mạng nơ-ron PyTorch, lưu trữ SQLite, đồ thị tri thức networkx hoặc Neo4j.
- Quy mô lúc viết báo cáo: 60 commit, 28 tệp test với 488 test tự động, bộ benchmark 50 ca.

1.2. Xuất phát điểm

Ý tưởng ban đầu là một sơ đồ ba khối vẽ trên bảng: dữ liệu có sẵn đi qua học sâu để **dự đoán**; kho tri thức (sách, web) đi qua RAG và agent để **trả lời**; học tăng cường kết hợp luật để **khuyên**. V1 và V2 hiện thực hai khối đầu và phần luật của khối thứ ba. V3 thêm phần học tăng cường, và nâng kho tri thức từ một kho tài liệu (knowledge base, chỉ là các đoạn văn để tìm kiếm) lên một đồ thị tri thức (knowledge graph) có nút, cạnh và nguồn gốc rõ ràng.

Dự án được xây theo từng phiên bản:

Phiên bản	Nội dung chính
V1	Predictor (MLP), RAG, Advisor qua Ollama, Rules, giao diện, CLI, SQLite, benchmark
V2	Agent dinh dưỡng, bộ nhớ dài hạn, tóm tắt lịch sử, truy hồi lai BM25 (tuỳ chọn), chạy benchmark trên Kaggle
V2.1	Agent xét nghiệm (ADA 2024), xử lý câu chào, giao diện trên Kaggle qua đường hầm Cloudflare
V2.2	Hiện chữ dần có lọc an toàn, vết thực thi từng agent, trang Hoạt động agent với sơ đồ động
V3	Đồ thị tri thức và Graph agent (networkx hoặc Neo4j), agent học tăng cường (Q-learning tuyến tính), OCR cho PDF dạng ảnh, hợp đồng dữ liệu giữa các agent

Phiên bản	Nội dung chính
V3.1	Đồ thị tri thức bật mặc định, ghép bằng chứng gen chung từ PrimeKG, thử nghiệm RGCN trên Kaggle, khung "Đồ thị tri thức đã dùng" trên giao diện
V3.2	Agent RL viết lại thành contextual bandit học từ 👍 / 👎 của người dùng, bật mặc định, khởi động trên phân bố BRFSS thật, có mặt nạ an toàn, ghi propensity và đánh giá SNIPS

1.3. Mục tiêu và phạm vi

Mục tiêu chức năng:

1. Ước lượng nguy cơ tiểu đường từ hồ sơ lối sống.
2. Trả lời câu hỏi về ăn uống, vận động, triệu chứng, xét nghiệm, có trích nguồn tài liệu y khoa.
3. Nhớ những gì người dùng đã khai (chẩn đoán, thuốc đang dùng, dị ứng) qua nhiều phiên.
4. Đọc kết quả xét nghiệm từ tệp hoặc văn bản và đối chiếu với ngưỡng tham khảo.
5. Tra thành phần dinh dưỡng các món ăn Việt.
6. Chuyển người dùng đi khám khi có dấu hiệu nguy cơ, triệu chứng hoặc cấp cứu.
7. Với mỗi yếu tố nguy cơ người dùng nhắc tới, cho biết tài liệu nói gì, dữ liệu đo được gì và hai nguồn có khớp nhau không.

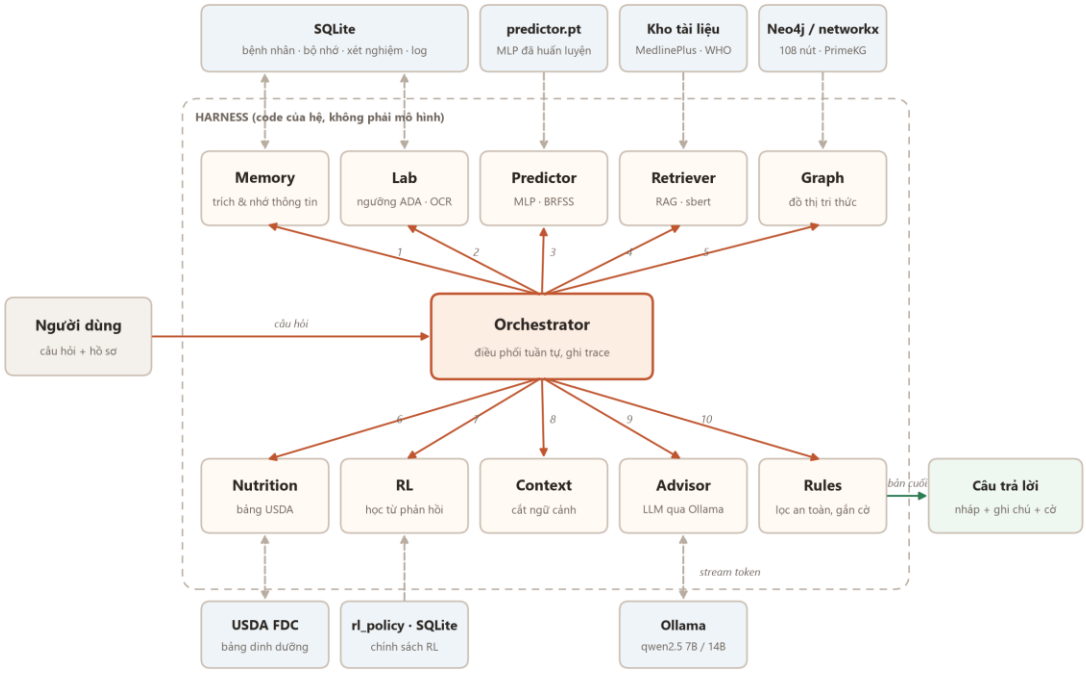
Mục tiêu phi chức năng:

- **An toàn**: không bao giờ hiển thị lời khuyên tự ý thay đổi thuốc hoặc insulin.
- **Quan sát được**: mọi lượt trả lời đều xem lại được từng bước, số token, thời gian.
- **Chạy được trên phần cứng miễn phí**: máy cá nhân có GPU 4 GB, hoặc GPU T4 của Kaggle.
- **Kiểm thử được**: mọi phần không phải mô hình ngôn ngữ đều có unit test.

Ngoài phạm vi: chẩn đoán, kê đơn, câu hỏi y khoa không liên quan tới tiểu đường.

2. Kiến trúc

2.1. Sơ đồ tổng thể

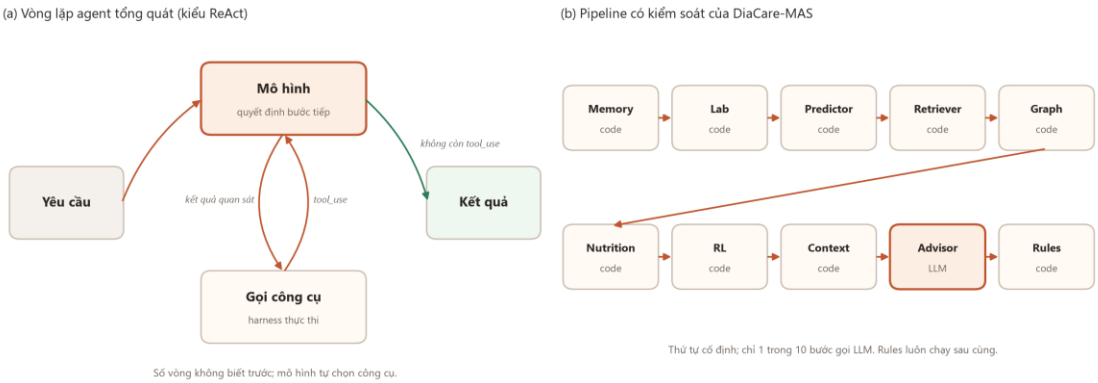


Kiến trúc DiaCare-MAS. Khung nét đứt là harness (phần code bao quanh mô hình). Số trên mũi tên là thứ tự Orchestrator gọi các agent.

Kiến trúc theo mẫu orchestrator-worker. Orchestrator giữ một đối tượng trạng thái `State` cho mỗi lượt hỏi, gọi lần lượt mười agent, mỗi agent đọc phần trạng thái nó cần và ghi kết quả vào đó. Các tài nguyên nằm ngoài khung harness: cơ sở dữ liệu SQLite, tệp mô hình đã huấn luyện, kho tài liệu, đồ thị tri thức (Neo4j hoặc tệp), chính sách học tăng cường, bảng dinh dưỡng và máy chủ Ollama.

Agent nào chạy được bật bằng biến `DIACARE_FEATURES`. Mặc định bật cả mười agent, trừ truy hồi lai BM25. Agent bị tắt thì Orchestrator bỏ qua, không ảnh hưởng các agent còn lại.

2.2. Vì sao là pipeline cố định chứ không phải vòng lặp agent



(a) Vòng lặp agent tổng quát kiểu ReAct, mô hình tự chọn công cụ. (b) Pipeline có kiểm soát của DiaCare-MAS.

Kiểu agent phổ biến hiện nay cho mô hình tự quyết định gọi công cụ nào ở mỗi bước. DiaCare-MAS cố ý không làm vậy, vì ba lý do:

1. Mô hình 7B chạy cục bộ chọn công cụ không ổn định. Thứ tự cố định loại bỏ hẳn lớp lỗi "chọn sai công cụ".
2. Những việc cần chính xác tuyệt đối (đổi đơn vị xét nghiệm, so ngưỡng, tính nguy cơ) được làm bằng code có test, không giao cho mô hình.
3. Agent kiểm tra an toàn phải luôn chạy cuối cùng. Trong vòng lặp tự do, mô hình có thể kết thúc mà không qua bước kiểm tra.

Kết quả là trong mười bước chỉ một bước gọi mô hình ngôn ngữ (Advisor). Bước tóm tắt lịch sử của Memory cũng gọi mô hình, nhưng chỉ khi hội thoại dài quá ngân sách.

2.3. Luồng xử lý một câu hỏi

Lấy ví dụ người dùng có hồ sơ nguy cơ cao, đã nhập phiếu xét nghiệm, hỏi: "Dạo này tôi khát nước liên tục, đi tiểu nhiều, nước tiểu có kiến bâu. Kết quả xét nghiệm của tôi thế nào?"

1. **Orchestrator** kiểm tra hồ sơ hợp lệ, kiểm tra đây không phải câu chào.
2. **Memory** trích sự kiện mới từ câu hỏi (không có), nhớ lại các sự kiện cũ của bệnh nhân.
3. **Lab** lấy kết quả mới nhất đã lưu (HbA1c 8,5%, đường huyết lúc đói 180 mg/dL, OGTT 182 mg/dL) và xếp mức: hai chỉ số ở ngưỡng tiểu đường, một chỉ số ở mức tiền tiểu đường.
4. **Predictor** chạy MLP trên 21 chỉ số hồ sơ: 92,1%, mức cao.
5. **Retriever** tìm được 4 đoạn tài liệu từ MedlinePlus và WHO.
6. **Graph** không thấy yếu tố nguy cơ nào trong câu hỏi (câu này nói về triệu chứng), nên không thêm gì.
7. **Nutrition** nhận ra đây không phải câu hỏi dinh dưỡng, bỏ qua.
8. **RL** thấy xét nghiệm ở ngưỡng tiểu đường và câu hỏi có triệu chứng, nên mặt nạ an toàn buộc chọn trọng tâm "khám và xét nghiệm xác nhận sớm", không thăm dò.
9. **Context** ghép ngữ cảnh cho Advisor, khoảng 1.000 token trên cửa sổ 4.096.
10. **Advisor** gọi Ollama, nhận bản nháp từng đoạn và đẩy lên giao diện theo từng câu đã qua kiểm tra.
11. **Rules** kiểm tra bản nháp, gắn ba cờ `high_risk_referral`, `lab_diabetes_range`, `symptom_referral` kèm ba ghi chú chuyển khám, rồi thêm lời nhắc bác sĩ.

Toàn bộ các bước được ghi vào vết thực thi và lưu trong cơ sở dữ liệu. Nếu người dùng hỏi tiếp "Tôi bị béo phì và huyết áp cao, có dễ bị tiểu đường không?", bước Graph nhận ra hai yếu tố `OBESITY` và `HIGH_BP`, và ngữ cảnh gửi cho Advisor tăng từ khoảng 940 lên 1.120 token (số đo ở chế độ demo). Dưới câu trả lời, người dùng có thể bấm 👍 / 👎 cho trọng tâm mà RL đã chọn.

3. Thiết kế chi tiết từng thành phần

3.1. Trạng thái dùng chung

`State` (tệp `state.py`) là một dataclass chứa mọi thứ của một lượt hỏi: câu hỏi, hồ sơ, lịch sử, mức nguy cơ, tài liệu, tri thức từ đồ thị (`kg`), trọng tâm do RL chọn (`rl`), kết quả dinh dưỡng, bộ nhớ, xét nghiệm,

bản nháp, câu trả lời cuối, cờ, số token đã dùng và vết thực thi. Các agent không gọi nhau trực tiếp mà chỉ đọc ghi trên `State`, nên thêm hay bỏ một agent không làm ảnh hưởng các agent khác.

3.2. Orchestrator

Tệp `orchestrator.py`. Hàm `run(state, on_token=None, on_step=None)` chạy pipeline và trả về `state`. Hai tham số tuỳ chọn dành cho giao diện: `on_token` nhận từng đoạn chữ khi mô hình đang viết, `on_step` nhận mỗi bước agent ngay khi bước đó xong.

Một số quy tắc điều phối:

- **Câu chào hỏi** ("chào bạn", "cảm ơn nhé") được trả lời ngay bằng câu soạn sẵn, không dự đoán nguy cơ, không gọi mô hình.
- **Lời nhắc nguy cơ cao** chỉ xuất hiện một lần mỗi phiên, trừ khi người dùng hỏi thẳng về nguy cơ, xét nghiệm hay đi khám.
- **Ghi chú xét nghiệm** chỉ nhắc khi người dùng vừa đưa số liệu mới hoặc chưa được nhắc lần nào trong phiên.
- **Lỗi bất kỳ** ở một agent làm lượt đó trả về thông báo sự cố với cờ `system_error`, ghi tên lỗi vào log, không làm sập giao diện.

Lớp `_Tracer` ghi mỗi bước thành `{agent, detail, ms}`. Lỗi trong hàm `on_step` của giao diện bị nuốt lại để không làm hỏng pipeline.

3.3. Memory: bộ nhớ ngắn hạn và dài hạn

Tệp `memory.py`.

Bộ nhớ dài hạn lưu năm loại sự kiện: chẩn đoán, thuốc đang dùng, dị ứng, chế độ ăn, bệnh kèm theo. Hàm `extract_facts` trích bằng biểu thức chính quy và chỉ nhận câu khai báo ngôi thứ nhất. "Mẹ tôi bị tiểu đường" không được ghi thành chẩn đoán của người dùng, vì từ chỉ người thân bị thay bằng một từ giả trước khi so khớp. Hàm chạy hai lượt, trên chữ có dấu và chữ đã bỏ dấu. Sự kiện được lưu vào bảng `memories`, hiển thị ở thanh bên và xoá được (xoá mềm, `active = 0`).

Thôi ghi nhớ khi thông tin hết đúng. Hàm `extract_retractions` nhận câu ngôi thứ nhất kiểu "tôi đã ngừng metformin", "mình bỏ uống insulin", "tôi không dùng insulin nữa" và tắt mục thuốc tương ứng; vết thực thi ghi "thôi ghi nhớ metformin". Câu hỏi như "tôi ngừng metformin được không?" hay câu về người thân ("mẹ tôi ngừng metformin") không làm tắt gì. Khai lại "tôi đang dùng metformin" thì mục đó được bật lại.

Khi đưa vào ngữ cảnh, bộ nhớ chiếm tối đa 150 token; nếu vượt thì bỏ sự kiện cũ nhất trước. Sự kiện "đã được chẩn đoán" còn được agent Rules dùng: với người đã chẩn đoán, hệ nhắc tái khám theo lịch thay vì khuyên đi xét nghiệm sàng lọc.

Bộ nhớ ngắn hạn là lịch sử phiên. Hàm `compress_history` giữ nguyên các tin nhắn mới nhất chiếm tối đa nửa ngân sách 800 token và nhờ mô hình tóm tắt phần cũ thành một tin nhắn hệ thống. Nếu tóm tắt lỗi, hệ dùng lịch sử gốc và để `build_context` cắt bớt như ở V1.

Bản tóm tắt được **lưu lại** vào bảng `summaries` cùng số tin đã phủ và một dấu nhận dạng (số tin và mã băm của tin cuối). Lượt sau có ba trường hợp:

Trường hợp	Làm gì	Gọi LLM
phần cũ vẫn đúng như lần trước	dùng lại bản đã lưu	không
phần cũ dài thêm vài tin	tóm tắt bản cũ cộng các tin mới (tăng dần)	có, ngắn
dấu nhận dạng không khớp (lịch sử bị cắt cửa sổ hay khác phiên)	tóm tắt lại từ đầu	có

Trước đây mỗi lượt của một hội thoại dài đều tóm tắt lại toàn bộ phần cũ; nay phần đã tóm tắt không bao giờ phải đưa lại cho mô hình. Dấu nhận dạng bảo đảm không dùng nhầm bản tóm tắt của một đoạn lịch sử khác.

3.4. Lab: đọc và xếp mức xét nghiệm

Tệp `labs.py`. Agent đọc bốn chỉ số: HbA1c, đường huyết lúc đói, đường huyết 2 giờ sau nghiệm pháp dung nạp glucose, đường huyết bất kỳ.

Nguồn đầu vào. Tệp `.txt`, `.csv`, `.json` (được làm phẳng), `.pdf`, ảnh chụp phiếu xét nghiệm, văn bản dán vào, hoặc giá trị gõ thẳng trong câu hỏi. Giá trị gõ trong chat được lưu với nguồn `chat`.

OCR (tệp `ocr.py`, V3). Trang PDF có lớp chữ được đọc bằng `pypdf`. Trang không có lớp chữ (bản scan, ảnh chụp) được kết xuất thành ảnh rồi chạy OCR. Có ba backend, dùng cái đầu tiên có trên máy: `rapidocr_onnxruntime`, Tesseract, hoặc bộ OCR có sẵn của Windows 10/11. Chữ sau OCR được chuẩn hoá cách viết các đơn vị hay bị đọc lệch (`%`, `mg/dL`, `mmol/L`) trước khi đưa vào bộ nhận dạng ở dưới.

Nhận dạng. Biểu thức chính quy chạy trên chữ đã bỏ dấu, nhận cả tên tiếng Việt lẫn tiếng Anh ("glucose lúc đói", "FPG", "đường huyết đói"). Một cửa sổ 60 ký tự sau tên chỉ số được quét tìm số đầu tiên hợp lệ, bỏ qua những số không phải kết quả như "75g" hay "2h" trong tên nghiệm pháp.

Chuẩn hoá đơn vị. Đường huyết mmol/L nhân 18 ra mg/dL. HbA1c theo chuẩn IFCC (mmol/mol) đổi sang % bằng $v/10{,}929 + 2{,}15$. Giá trị ngoài khoảng sinh lý hợp lý bị bỏ.

Xếp mức theo ADA 2024:

Chỉ số	Bình thường	Tiền tiểu đường	Ngưỡng tiểu đường
HbA1c	< 5,7%	5,7–6,4%	≥ 6,5%
Đường huyết lúc đói	< 100 mg/dL	100–125	≥ 126
OGTT 2 giờ	< 140 mg/dL	140–199	≥ 200
Đường huyết bất kỳ			≥ 200 kèm triệu chứng

Đường huyết dưới 70 mg/dL được xếp là hạ đường huyết, kèm lời dặn gọi 115 nếu lú lẫn hoặc ngất.

3.5. Predictor: dự đoán nguy cơ

Tệp `predictor.py`, `model.py`, huấn luyện bằng `train_predictor.py`.

- **Dữ liệu**: CDC Diabetes Health Indicators (BRFSS 2015), 253.680 mẫu, 21 chỉ số, tỉ lệ dương 13,9%. Script tải trực tiếp từ UCI (id 891).

- **Mô hình**: MLP 21 → 64 → 32 → 1, ReLU, dropout 0,2. Hàm mất mát `BCEWithLogitsLoss` có trọng số lớp dương để bù lệch lớp, tối ưu Adam tốc độ học 0,001, 15 epoch. Đầu vào được chuẩn hoá bằng `StandardScaler` lưu kèm mô hình.
- **Chia tập**: 80/20 phân tầng, hạt giống 42.

Mô hình	AUC	F1	Recall
Logistic Regression (cân bằng lớp)	0,820	0,441	0,761
MLP	0,827	0,443	0,803

Mức nguy cơ được chia ba: thấp dưới 0,2, trung bình đến dưới 0,5, cao từ 0,5. Xác suất chưa được hiệu chỉnh, nên lời dặn hệ thống cấm mô hình ngôn ngữ nêu con số phần trăm cho người dùng; chỉ giao diện kỹ thuật hiển thị số.

Hồ sơ không hợp lệ (thiếu trường, ngoài khoảng, không phải số nguyên) bị chặn trước khi tới Predictor, trả về cờ `invalid_input`.

3.6. Retriever: tìm tài liệu

Tệp `retriever.py`, `kb.py`.

- **Kho tài liệu**: năm tệp MedlinePlus và WHO trong thư mục `kb/`, mỗi tệp ghi nguồn và ngày tải.
- **Chia đoạn**: 600 ký tự, chồng lấp 100, cắt ở ranh giới đoạn văn, câu hoặc khoảng trắng. Bản đầu cắt cứng mỗi 600 ký tự đã tạo ra những mẩu chữ bị cắt đôi (“has” thành “ha” rồi khớp nhầm “Hà Nội”).
- **Embedding**: `paraphrase-multilingual-MiniLM-L12-v2`, nên câu hỏi tiếng Việt tìm được tài liệu tiếng Anh. Điểm cosine dưới 0,2 bị bỏ để câu ngoài chủ đề không kéo về tài liệu rác.
- **Truy hồi lai** (tuỳ chọn): BM25 kết hợp với dense bằng Reciprocal Rank Fusion, $k_{\text{RRF}} = 60$.

Đánh giá trên 24 câu đúng chủ đề và 4 câu ngoài chủ đề:

Chế độ	recall@1	recall@4	MRR	Câu ngoài chủ đề trả về rỗng
dense, ngưỡng 0,2	0,708	0,917	0,785	0,50
BM25	0,333	0,500	0,410	0,50
lai RRF, ngưỡng 0,2	0,750	0,917	0,809	0,25

Truy hồi lai nhỉnh hơn một câu ở recall@1 nhưng lọc câu ngoài chủ đề kém hơn rõ, nên mặc định dùng dense.

3.7. Graph: đồ thị tri thức

Tệp `kg.py`. Kho tài liệu ở mục 3.6 chỉ là các đoạn văn: Retriever tìm được đoạn gần câu hỏi, nhưng hệ không biết đoạn đó khẳng định điều gì, về yếu tố nào, và khẳng định ấy có được dữ liệu ủng hộ không. Đồ thị tri thức lưu tri thức thành nút và cạnh có kiểu, mỗi cạnh ghi rõ nguồn, nên hệ đem được các nguồn ra đối chiếu với nhau.

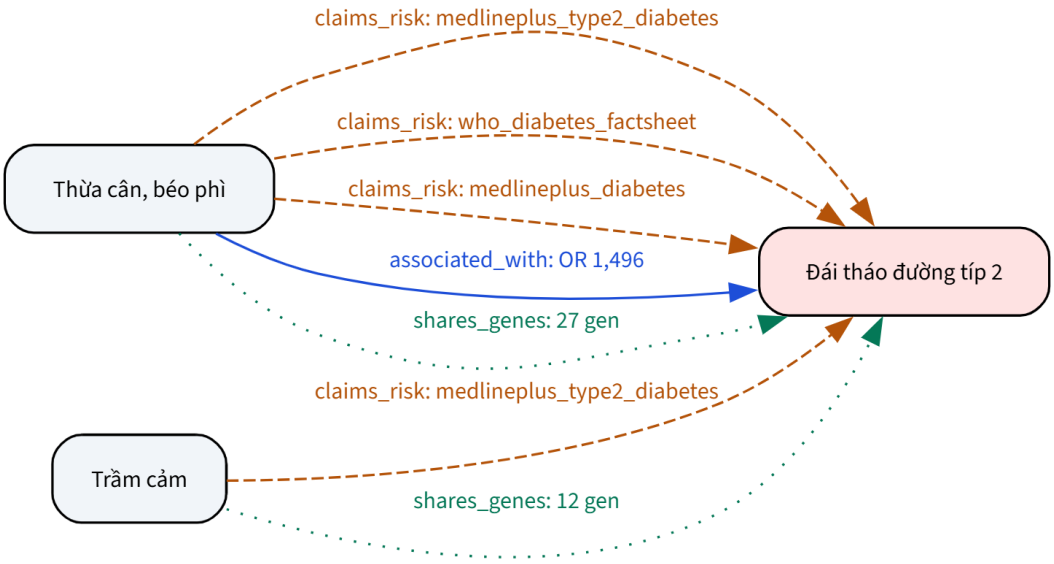
Cấu trúc

Đồ thị có 108 nút thuộc 7 nhãn và 228 cạnh thuộc 6 quan hệ.

Nhãn nút	Số nút	Ví dụ
`Condition`	5	đái tháo đường típ 2, tiền đái tháo đường, hạ đường huyết
`RiskFactor`	17	béo phì, tăng huyết áp, trầm cảm, tiền sử gia đình
`LabTest`	4	HbA1c, đường huyết lúc đói
`Symptom`	27	khát nước, kiến bâu, hôn mê
`Action`	2	đi khám bác sĩ, gọi cấp cứu 115
`Food`	50	cơm trắng, xôi, mướp đắng
`Nutrient`	3	tinh bột, đường, chất xơ

Quan hệ	Số cạnh	Nghĩa	Lấy từ đâu
`indicates`	13	xét nghiệm → ngưỡng	ngưỡng ADA 2024 trong `labs.py`
`requires`	27	triệu chứng → hành động	danh sách triệu chứng và cấp cứu trong `rules.py`
`contains`	143	món ăn → chất dinh dưỡng	bảng USDA `vn_foods.json`
`claims_risk`	28	yếu tố → ĐTĐ típ 2: **tài liệu nói**	câu trích từ tài liệu MedlinePlus, WHO trong `kb/`
`associated_with`	9	yếu tố → ĐTĐ típ 2: **dữ liệu đo**	hồi quy logistic trên BRFSS 2015 (`tools/build_kg_stats.py`)
`shares_genes`	8	yếu tố → ĐTĐ típ 2: **gen chung**	PrimeKG 2.1 (`tools/primekg_subgraph.py`)

Ba quan hệ đầu là dữ liệu tra cứu, hệ coi là đúng. Ba quan hệ sau đều nối một yếu tố nguy cơ tới nút đái tháo đường típ 2, nhưng có độ tin cậy khác nhau nên được giữ thành **ba loại cạnh song song**, không gộp thành một cạnh “có liên quan”.



Đồ thị con mà Graph agent đọc cho câu hỏi "tôi bị béo phì và trầm cảm": ba loại cạnh song song từ mỗi yếu tố tới đái tháo đường típ 2.

Ba nguồn cho cùng một cặp nút

Tài liệu nói (`claims_risk`). Hàm `extract_claims` đọc các tài liệu trong `kb/`, tìm câu nằm trong danh sách "bạn có nguy cơ cao hơn nếu..." của đái tháo đường típ 2, hoặc câu về phòng ngừa, rồi nối câu vào nút yếu tố bằng danh sách từ đồng nghĩa. Mỗi cạnh lưu tên tệp, câu gốc và chiều (tăng hay giảm nguy cơ). Danh sách nguy cơ của đái tháo đường thai kỳ bị bỏ qua, và câu kiểu "người tiểu đường dễ bị đột quỵ" không được coi là yếu tố nguy cơ vì chiều quan hệ ngược lại (đó là biến chứng). Kết quả: 28 khẳng định.

Dữ liệu đo (`associated_with`). Với 9 yếu tố có cột tương ứng trong BRFSS 2015 (253.680 người), script tính hai tỉ số chênh (OR): `or_marginal` khi xét riêng yếu tố đó, và `or_adjusted` trong mô hình logistic có đủ 21 chỉ số. Ví dụ tăng huyết áp có OR 5,04 khi xét riêng và 2,13 sau hiệu chỉnh; hút thuốc 1,42 khi xét riêng nhưng 0,99 sau hiệu chỉnh, nghĩa là phần lớn mối liên hệ đến từ các yếu tố đi kèm. Đây là liên hệ trong dữ liệu khảo sát cắt ngang, không phải nhân quả, và câu gửi cho mô hình ngôn ngữ nói rõ điều này.

Gen chung (`shares_genes`). Với các yếu tố mà BRFSS không đo (trầm cảm, buồng trứng đa nang...), hệ cần một nguồn độc lập ngoài lời tài liệu. PrimeKG là đồ thị tri thức y sinh của Harvard (Chandak và cộng sự, 2023) với khoảng 129 nghìn nút và 8 triệu cạnh. Script lấy các gen mà PrimeKG nối với cả yếu tố lẫn đái tháo đường típ 2 (đái tháo đường típ 2 có 221 gen). Kết quả: bệnh tim chung 29 gen, béo phì 27, trầm cảm 12, đột quỵ 8, rối loạn mỡ máu 6, tăng huyết áp 4, buồng trứng đa nang 4, hút thuốc 1. Số gen chung chỉ nói hai bệnh **có chung cơ chế sinh học**, không phải mức nguy cơ, và câu đưa vào ngữ cảnh ghi đúng như vậy. Chỉ tệp kết quả nhỏ `kb/graph/primekg_shared_genes.json` nằm trong repo; dữ liệu PrimeKG gốc (khoảng 400 MB) không cần khi chạy ứng dụng.

Kiểm chứng tài liệu bằng dữ liệu

Hàm `verify_claim` đem mỗi cạnh `claims_risk` đối chiếu với cạnh `associated_with` cùng cặp nút và cho một trong bốn kết luận:

Kết luận	Khi nào	Số khẳng định

Kết luận	Khi nào	Số khẳng định
nhất quán	OR cùng chiều với tài liệu	15
mâu thuẫn	OR ngược chiều với tài liệu	0
không đủ bằng chứng	OR nằm trong khoảng [1/1,1; 1,1]	0
không kiểm chứng được	BRFSS không có cột cho yếu tố này	13

Không khẳng định nào mâu thuẫn với dữ liệu. 13 khẳng định không kiểm chứng được (tiền sử gia đình, chủng tộc, đái tháo đường thai kỳ, buồng trứng đa nang...) được đánh dấu rõ trong ngữ cảnh thay vì bị bỏ qua lặng lẽ. Với "ít vận động", BRFSS đo chiều ngược (`PhysActivity`, có vận động), nên OR nhỏ hơn 1 lại là kết luận nhất quán.

Graph agent khi trả lời

Hàm `link_factors` nối câu hỏi tiếng Việt (có dấu hay không dấu) vào nút yếu tố nguy cơ. Với tối đa 3 yếu tố, agent lấy cả ba loại cạnh của từng yếu tố và viết mỗi yếu tố thành một dòng, ví dụ:

> Thừa cân, béo phì: tài liệu (medlineplus_diabetes, medlineplus_type2_diabetes, who_diabetes_factsheet) nêu là yếu tố nguy cơ. BRFSS 2015 (n = 253.680): OR 1,681 khi xét riêng, 1,496 khi hiệu chỉnh 21 chỉ số (mỗi độ lệch chuẩn); là liên hệ, không phải nhân quả. PrimeKG: chung 27 gen với ĐTĐ típ 2 (INS, LEPR, FGF21), là cơ chế chung, không phải mức nguy cơ.

Khối này có nhãn nguồn `[kg]` và ngân sách 300 token. Lúc đầu ngân sách là 250; sau khi thêm dòng gen chung, một câu hỏi nhắc ba yếu tố đo được 283 token và bị cắt mất dòng thứ ba, nên ngân sách được nâng lên 300 (câu ba yếu tố hiện dùng 268).

Hai nơi lưu: networkx và Neo4j

Mặc định đồ thị được dựng trong bộ nhớ bằng networkx từ các tệp nguồn, nên test, chế độ demo và Kaggle không cần máy chủ. Đặt `DIACARE_KG=neo4j` thì Graph agent truy vấn Neo4j 5 qua giao thức bolt. Script `tools/kg_neo4j.py` (hoặc `chay_kg.bat`) bật Neo4j trong Docker và nạp đồ thị từ tệp `DB_diabet.cypher`. Tệp này do `to_cypher()` sinh ra và chỉ dùng `MERGE`, nên chạy lại nhiều lần không tạo nút trùng. Mỗi yếu tố được lấy bằng một câu Cypher:

```
MATCH (f:RiskFactor {id: $fid})
OPTIONAL MATCH (f)-[a:associated_with]->(:Condition {id: 'T2D'})
OPTIONAL MATCH (f)-[s:shares_genes]->(:Condition {id: 'T2D'})
OPTIONAL MATCH (f)-[c:claims_risk]->(:Condition {id: 'T2D'})
RETURN f.name AS name, f.measurable AS measurable,
       a {.or_marginal, .or_adjusted, .unit, .n} AS association,
       s {.n_genes, .genes, .primekg_nodes} AS genes,
       collect(c {.source, .direction, .verdict}) AS claims
```

Với câu "Tôi bị béo phì, huyết áp cao và trầm cảm", hai backend cho ra khối ngữ cảnh giống nhau từng chữ. Nếu không kết nối được Neo4j, agent in cảnh báo và tự quay về networkx, không làm hỏng lượt trả lời.

Thử nghiệm RGCN trên PrimeKG

Đồ thị của DiaCare nhỏ (108 nút), nên các phép duyệt cạnh như trên là đủ; học biểu diễn đồ thị bằng mạng nơ-ron không có lợi ở quy mô này. Để thử phương pháp học trên đồ thị quan hệ, dự án cắt từ PrimeKG một đồ thị con quanh 68 bệnh đái tháo đường (lấy các nút cách 1 bước). Loại cạnh thuốc–

thuốc bị bỏ vì chiếm khoảng 80% số cạnh mà không liên quan bài toán. Kết quả là đồ thị con 2.818 nút, 28.781 cạnh, 20 loại quan hệ, chia 80/10/10 theo cặp nút.

Mô hình RGCN (Schlichtkrull và cộng sự, 2018) hai tầng, phân rã cơ sở 10 ma trận, đầu ra chấm điểm bằng DistMult, viết bằng PyTorch thuần (`tools/kg_rgcn.py`). Mốc so sánh là DistMult không có tầng đồ thị (Yang và cộng sự, 2015). Huấn luyện trên GPU T4 của Kaggle, 600 epoch, 3 hạt giống, chấm theo MRR và Hits@k có lọc trên tập test:

Mô hình	MRR	Hits@1	Hits@10	Giây
DistMult	0,395 ± 0,001	0,286	0,618	34
RGCN 2 tầng	0,407 ± 0,005	0,304	0,622	61

RGCN hơn DistMult một chút (MRR +0,012): chênh lệch lớn hơn độ lệch giữa các hạt giống nhưng nhỏ. Theo từng quan hệ, mô hình dự đoán tốt cạnh chỉ định thuốc (MRR 0,77), chống chỉ định (0,76) và bệnh–protein (0,65), nhưng gần như không học được protein–protein (0,03) và bệnh–bệnh (0,11).

Dự án cũng thử dùng độ gần embedding giữa yếu tố DiaCare và nút đái tháo đường làm thêm một loại bằng chứng. Kết quả không ổn định: phân vị của cùng một cặp nút lệch nhau tới 0,43 giữa các hạt giống, và thứ tự các yếu tố không khớp hiểu biết lâm sàng. Vì vậy **ứng dụng không dùng embedding RGCN**; chỉ số gen chung (phép đếm trực tiếp trên PrimeKG, tái lập được) được đưa vào đồ thị.

3.8. Nutrition: dinh dưỡng món ăn

Tệp `nutrition.py`, dữ liệu `data/nutrition/vn_foods.json` gồm 50 món ăn và nguyên liệu Việt với năng lượng, carbohydrate, chất xơ, đường trên 100 g, quy đổi từ USDA FoodData Central, kèm khẩu phần ước lượng.

Agent nhận món ăn trong câu hỏi, ước lượng khẩu phần, tạo bảng và đưa vào ngữ cảnh kèm nguồn `[usda_fdc]`. Bảng chỉ được đưa vào khi câu hỏi thực sự là câu hỏi dinh dưỡng.

Khó khăn chính là từ đa nghĩa. Lần chạy benchmark thứ ba cho thấy 7 trên 18 câu có bảng dinh dưỡng bị chèn sai: "tiểu đường" bị nhận thành món *đường*, "giá vàng" thành *giá đỗ*. Bản sửa xét hai từ trước và sau (đã bỏ dấu) của các từ một âm tiết dễ trùng nghĩa: đường (tiểu đường, đường huyết), giá (giá vàng, đánh giá), cá (cá nhân), cam (cam kết), lạc (lạc quan, liên lạc), nhãn (nhãn thuốc), vải.

3.9. RL: học chọn trọng tâm tư vấn từ phản hồi

Tệp `rl.py`, bật mặc định. Cùng một hồ sơ có thể vừa thừa cân, vừa ít vận động, vừa có xét nghiệm xấu; một câu trả lời ngắn không nói kỹ được hết. Agent RL chọn **một trong sáu trọng tâm** (khám và xét nghiệm sớm, theo dõi đường huyết, cân nặng và khẩu phần, vận động, huyết áp và mỡ máu, duy trì lối sống), đưa một câu hướng dẫn vào ngữ cảnh của Advisor (ngân sách 120 token), rồi **học từ việc người dùng chấm trọng tâm đó**.

Bài toán

Mỗi lượt hỏi là một vòng tương tác:

1. Agent quan sát trạng thái s, mã hoá thành vector 13 đặc trưng $\phi(s)$: mức nguy cơ (3), xét nghiệm báo động, tiền tiểu đường, đã chẩn đoán, thừa cân, ít vận động, tim mạch hoặc hút thuốc, chủ đề triệu chứng/xét nghiệm, chủ đề lối sống, và "nguy cơ cao mà chưa chẩn đoán".

2. Agent chọn hành động a (một trọng tâm).
3. Dưới câu trả lời, giao diện hỏi "Trọng tâm này có hợp với bạn không?". Người dùng bấm 👍 hoặc 👎, tức phần thưởng $r = +1$ hoặc -1.
4. Agent cập nhật chính sách bằng đúng cặp (s, a, r) đó.

Agent chỉ biết phần thưởng của hành động nó đã chọn, không biết hành động khác sẽ được chấm thế nào. Đó là học tăng cường với phản hồi đánh giá, khác học có giám sát (Sutton và Barto, chương 2). Hệ không quan sát được sức khoẻ của người dùng thay đổi ra sao sau lời khuyên, nên không có bước chuyển $s \to s'$ đáng tin để ước lượng giá trị dài hạn. Vì vậy bài toán được đặt đúng là **contextual bandit** (học tăng cường một bước, $\gamma = 0$), dạng RL được dùng nhiều nhất trong các hệ khuyến nghị chạy thật. Bản trước dùng Q-learning với $\gamma = 0{,}8$ trên một môi trường mô phỏng tự đặt; cách đó bị bỏ vì bước chuyển trạng thái là bịa, và vì thang điểm của nó (Q khoảng 3 đến 5) khiến một cú 👍 (+1) lại *hạ* điểm hành động vừa được khen.

Mô hình và cập nhật

Mỗi hành động có một mô hình hồi quy ridge tuyến tính dự đoán phần thưởng, như LinUCB (Li và cộng sự, 2010):

$$\hat{r}_a(s) = w_a^{\top}\phi(s), \qquad w_a = A_a^{-1}b_a, \qquad A_a = \lambda I + \sum \phi\phi^{\top}, \quad b_a = \sum r\,\phi$$

Một phản hồi chỉ cộng $\phi\phi^{\top}$ vào A_a và $r\phi$ vào b_a, nên học trực tuyến tức thì, không cần huấn luyện lại. Mô hình sau mỗi phản hồi được lưu vào bảng `rl_model` của SQLite; mở lại ứng dụng thì học tiếp từ chỗ cũ.

Chọn hành động theo ba lớp:

1. **Mặt nạ an toàn.** Xét nghiệm ở ngưỡng tiểu đường hoặc hạ đường huyết, hoặc câu hỏi có triệu chứng, cấp cứu: luôn chọn "khám sớm", xác suất 1, không bao giờ thăm dò.
2. **Epsilon-greedy.** Ngoài trường hợp trên, với xác suất $1 - \varepsilon$ chọn hành động điểm cao nhất (khai thác), với xác suất $\varepsilon = 0{,}1$ chọn ngẫu nhiên một trong sáu (thăm dò). Cả sáu trọng tâm đều là lời khuyên an toàn, nên thăm dò không gây hại. Trọng tâm vừa dùng ở lượt trước bị trừ 0,25 điểm để lời khuyên không lặp mãi.
3. **Rules** vẫn chạy sau cùng và quyết định câu trả lời cuối; RL không ghi đè được.

Mỗi quyết định được ghi vào bảng `rl_events` cùng trạng thái, hành động, **xác suất đã chọn hành động đó** (propensity), cách chọn (mặt nạ, khai thác, thăm dò) và phiên bản chính sách. Mỗi sự kiện chỉ được chấm và học một lần. Benchmark đặt $\varepsilon = 0$ để kết quả tái lập được.

Khởi động trước khi có phản hồi

Lúc chưa có người dùng nào chấm, chính sách xuất phát từ một **tiên nghiệm**: điểm ưu tiên lâm sàng do nhóm đặt (ví dụ "khám sớm" được +1 khi xét nghiệm ở ngưỡng tiểu đường, "vận động" được điểm cao khi người ít vận động hỏi về tập luyện), quy về thang $[-1{,}1]$ giống 👍 / 👎. Trạng thái để khởi động không còn bịa ngẫu nhiên:

- **Hồ sơ** lấy theo tần suất thật của 253.680 người BRFSS 2015: 24 tổ hợp (mức nguy cơ do chính Predictor tính, thừa cân, ít vận động, tim mạch). Tệp nhỏ `kb/rl/brfss_state_freq.json` do `tools/build_rl_states.py` sinh ra.

- **Chủ đề câu hỏi** lấy theo 50 câu hỏi của bộ benchmark: chung 38%, dinh dưỡng 32%, triệu chứng 18%, xét nghiệm 8%, vận động 4%.
- **Xét nghiệm và chẩn đoán** BRFSS không có, nên dùng tỉ lệ giả định và ghi rõ trong tệp.

Độ nặng của tiên nghiệm bằng 30 lượt phản hồi cho mỗi hành động, nên phản hồi thật dần lấn át nó. Trên 5.000 trạng thái mới (hạt giống khác lúc huấn luyện), điểm tiên nghiệm trung bình:

Chính sách	Điểm TB
chọn ngẫu nhiên	0,173
luôn khuyên khám	0,300
chính sách khởi động	**0,893**
chọn tối ưu (cận trên)	0,917

Bảng này chỉ cho thấy mô hình tuyến tính học được tiên nghiệm (đạt 97% cận trên); nó **không** chứng minh lời khuyên tốt hơn cho người thật, vì tiên nghiệm do nhóm đặt. Bản đầu thiếu đặc trưng "nguy cơ cao mà chưa chẩn đoán" và chỉ đạt 0,834: mô hình tuyến tính xếp "huyết áp" trên "khám sớm" cho người nguy cơ cao chưa chẩn đoán, nên đặc trưng tương tác này được thêm vào.

Đánh giá từ phản hồi thật

Vì mỗi quyết định có propensity, chính sách hiện tại được đánh giá lại trên các lượt đã ghi mà không cần cho nó chạy thật, bằng ước lượng IPS và SNIPS (Swaminathan và Joachims, 2015):

$$\hat{V}_{\text{SNIPS}}(\pi) = \frac{\sum_i \frac{\mathbb{1}[a_i = \pi(s_i)]}{p_i} r_i}{\sum_i \frac{\mathbb{1}[a_i = \pi(s_i)]}{p_i}}$$

Trang Thống kê hiện số quyết định, số phản hồi, phần thưởng trung bình đã nhận, ước lượng SNIPS và phần thưởng theo từng trọng tâm.

Một test mô phỏng người dùng chứng minh vòng học hoạt động: người dùng giả luôn chấm 👍 cho "vận động" và 👎 cho mọi trọng tâm khác ở một trạng thái mà chính sách khởi động chọn "cân nặng". Sau 200 lượt (có thăm dò), chính sách tham lam chuyển sang "vận động", và SNIPS của chính sách mới cao hơn phần thưởng trung bình đã ghi.

Lúc viết báo cáo, hệ **chưa có phản hồi của người dùng thật** (0 lượt). Mọi con số về chất lượng ở trên đều đến từ tiên nghiệm hoặc người dùng mô phỏng.

3.10. Context: ngân sách token

Tệp `context.py`. Hàm `build_context(agent, state)` trả về lát ngữ cảnh riêng cho từng agent. Với Advisor, các hằng số ngân sách:

Hằng số	Giá trị	Ý nghĩa
`num_ctx`	4.096	cửa sổ ngữ cảnh gửi cho Ollama
`ANSWER_RESERVE_TOKENS`	512	chừa cho câu trả lời, bằng giới hạn `num_predict`

Hằng số	Giá trị	Ý nghĩa
SYSTEM_RESERVE_TOKENS	250	lời dặn hệ thống và nhãn
HISTORY_TOKEN_BUDGET	800	lịch sử hội thoại, lấy từ mới đến cũ
MEMORY_TOKEN_BUDGET	150	bộ nhớ dài hạn
KG_TOKEN_BUDGET	300	tri thức từ đồ thị, tối đa 3 yếu tố
RL_TOKEN_BUDGET	120	trọng tâm tư vấn do RL chọn
CHARS_PER_TOKEN	4,0	hệ số ước lượng; đo thật trên Kaggle khoảng 4,6

Phần còn lại được chia theo thứ tự ưu tiên: bộ nhớ, xét nghiệm, bảng dinh dưỡng, tri thức đồ thị, trọng tâm RL, rồi tài liệu truy hồi cho tới khi hết chỗ. Khối đồ thị vượt ngân sách thì bỏ bớt dòng cuối, không cắt giữa dòng. Ước lượng dùng 4,0 ký tự mỗi token, thấp hơn số đo thật, nên luôn dư chứ không thiếu. Nếu số token vào thực tế vượt 90% cửa sổ, hệ gắn cờ `context_near_limit` để cảnh báo câu trả lời có thể bị cắt.

Agent Rules chỉ nhận mức nguy cơ, bản nháp, câu hỏi, cờ đã chẩn đoán và xét nghiệm. Predictor chỉ nhận hồ sơ.

3.11. Advisor và mô hình ngôn ngữ

Tệp `advisor.py`, `llm.py`. Advisor ghép lời dặn hệ thống với khối bộ nhớ, khối xét nghiệm, mức nguy cơ (chỉ dạng chữ), bảng dinh dưỡng, khối tri thức đồ thị `[kg]`, trọng tâm RL, tài liệu có nhãn nguồn và câu hỏi. Lời dặn hệ thống yêu cầu:

- chỉ dùng tài liệu được cung cấp, trích nguồn dạng `[tên_tệp]`, nói rõ khi tài liệu không đủ;
- không kê đơn, không hướng dẫn thay đổi liều thuốc hoặc insulin;
- không nêu nguy cơ dạng phần trăm;
- khi có xét nghiệm: giải thích so với ngưỡng, trích `[ada_2024]`, nói rõ không phải chẩn đoán.

Lớp `OllamaLLM` gọi `/api/chat` với nhiệt độ 0,2, `num_ctx` 4.096 và `num_predict` 512. Giới hạn `num_predict` được thêm sau khi lần chạy Kaggle thứ hai gặp câu trả lời viết mãi tới hết thời gian chờ. Khi có `on_token`, lớp này bật chế độ stream và đọc từng dòng JSON của Ollama. Lớp `DemoLLM` là mô hình giả, trích nguyên đoạn tài liệu đầu tiên, dùng cho test và chạy thử giao diện không cần GPU.

3.12. Rules: rào chắn an toàn

Tệp `rules.py`. Agent làm bốn việc theo thứ tự:

1. **Chặn lời khuyên đổi thuốc.** Câu có động từ kiểu dừng, ngừng, bỏ, giảm liều, tăng liều đứng gần từ chỉ thuốc (insulin, metformin, thuốc) bị coi là nguy hiểm, trừ khi câu có phủ định, là câu cảnh báo, hoặc nói về thuốc lá. Nếu có, toàn bộ bản nháp được thay bằng câu “Mình không thể đưa ra lời khuyên về việc thay đổi thuốc hoặc insulin. Vui lòng hỏi bác sĩ điều trị của bạn.”
2. **Gắn ghi chú chuyển khám** theo nguy cơ, chẩn đoán, xét nghiệm.
3. **Nhận diện triệu chứng và cấp cứu** từ câu hỏi, gồm cả cách nói dân gian (“kiến bâu”, “đái nhiều”, “sụt ký”) và câu nói vòng (“vết thương ... không lành”, “nước tiểu ... kiến”).
4. **Thêm lời nhắc** “Đây chỉ là thông tin tham khảo, không thay thế tư vấn của bác sĩ.”

Danh sách cờ:

Cờ	Khi nào bật
`unsafe_advice_blocked`	bản nháp có lời khuyên thay đổi thuốc, đã bị thay
`high_risk_referral`	nguy cơ cao, chưa nhắc trong phiên hoặc người dùng hỏi về nguy cơ
`diagnosed_followup`	người dùng đã khai được chẩn đoán
`lab_diabetes_range`	có chỉ số ở ngưỡng tiểu đường
`lab_prediabetes_range`	có chỉ số ở mức tiền tiểu đường
`lab_low_glucose`	đường huyết dưới 70 mg/dL
`symptom_referral`	câu hỏi có triệu chứng
`emergency_referral`	câu hỏi có dấu hiệu cấp cứu
`small_talk`	câu chào hỏi, trả lời ngay
`invalid_input`	hồ sơ không hợp lệ
`system_error`	lỗi trong pipeline
`context_near_limit`	token vào vượt 90% cửa sổ
`history_summarized`	lịch sử đã được tóm tắt

Lọc khi stream. Lớp `SafeStream` gom các đoạn chữ của mô hình cho tới dấu kết thúc câu, kiểm tra cả câu, và chỉ đẩy lên giao diện những câu an toàn. Gặp câu nguy hiểm thì dừng hiển thị. Câu trả lời cuối vẫn do Rules quyết định, nên streaming không làm thay đổi kết luận an toàn nào.

4. Cơ sở dữ liệu

SQLite, tệp `db.py`. Chín bảng:

Bảng	Nội dung	Cột chính
`patients`	hồ sơ bệnh nhân	`name`, `profile_json`
`sessions`	phiên hội thoại	`patient_id`
`messages`	tin nhắn	`session_id`, `role`, `content`
`memories`	sự kiện dài hạn	`patient_id`, `kind`, `content`, `active`
`labs`	kết quả xét nghiệm	`patient_id`, `test`, `value`, `unit`, `source`
`summaries`	bản tóm tắt lịch sử, dùng lại giữa các lượt	`session_id`, `upto_message_index`, `content`, `anchor`
`runs`	log từng lượt chạy	`question`, `risk`, `flags_json`, `sources_json`, `draft`, `final`, `usage_json`, `error`, `trace_json`
`rl_events`	quyết định của agent RL và phản hồi	`state_json`, `action`, `propensity`, `mode`, `policy_version`, `reward`, `rated_at`
`rl_model`	mô hình RL đã học tới hiện tại (một dòng)	`model_json`

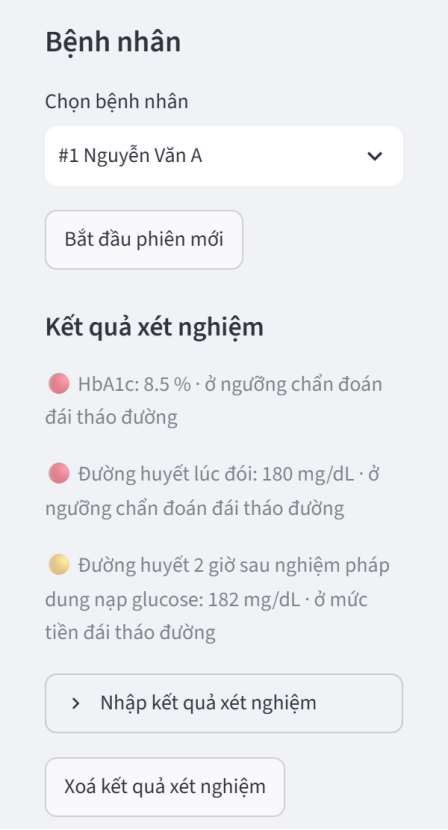
Bảng `runs` là nguồn dữ liệu cho các trang Lịch sử, Thống kê và Hoạt động agent. Cột `trace_json` được thêm ở V2.2, cột `messages.rl_event_id` ở V3.2 (nối câu trả lời với quyết định RL, để chấm và để phạt lặp); khi mở một cơ sở dữ liệu cũ, hệ tự thêm các cột này bằng `ALTER TABLE`. Song song với SQLite, mỗi lượt chạy còn được ghi một dòng JSON vào `logs/runs.jsonl`.

Nơi lưu thứ hai là **Neo4j**, dùng cho đồ thị tri thức khi đặt `DIACARE_KG=neo4j` (mục 3.7). Neo4j chạy trong container Docker riêng `diacare-neo4j`, cổng 7475 cho Neo4j Browser và 7688 cho bolt; dữ liệu nằm ở thư mục `neo4j_data/`. Mật khẩu được sinh ngẫu nhiên khi tạo container và lưu trong `neo4j.local.json`, tệp này không đưa lên repo. SQLite giữ dữ liệu của người dùng; Neo4j chỉ giữ tri thức chung, không có thông tin bệnh nhân nào.

5. Giao diện

Giao diện Streamlit (`src/app.py`) có bốn trang chọn bằng nút radio. Không dùng tab vì ô nhập chat của Streamlit chỉ ghim được xuống đáy màn hình khi nằm ở cấp ngoài cùng.

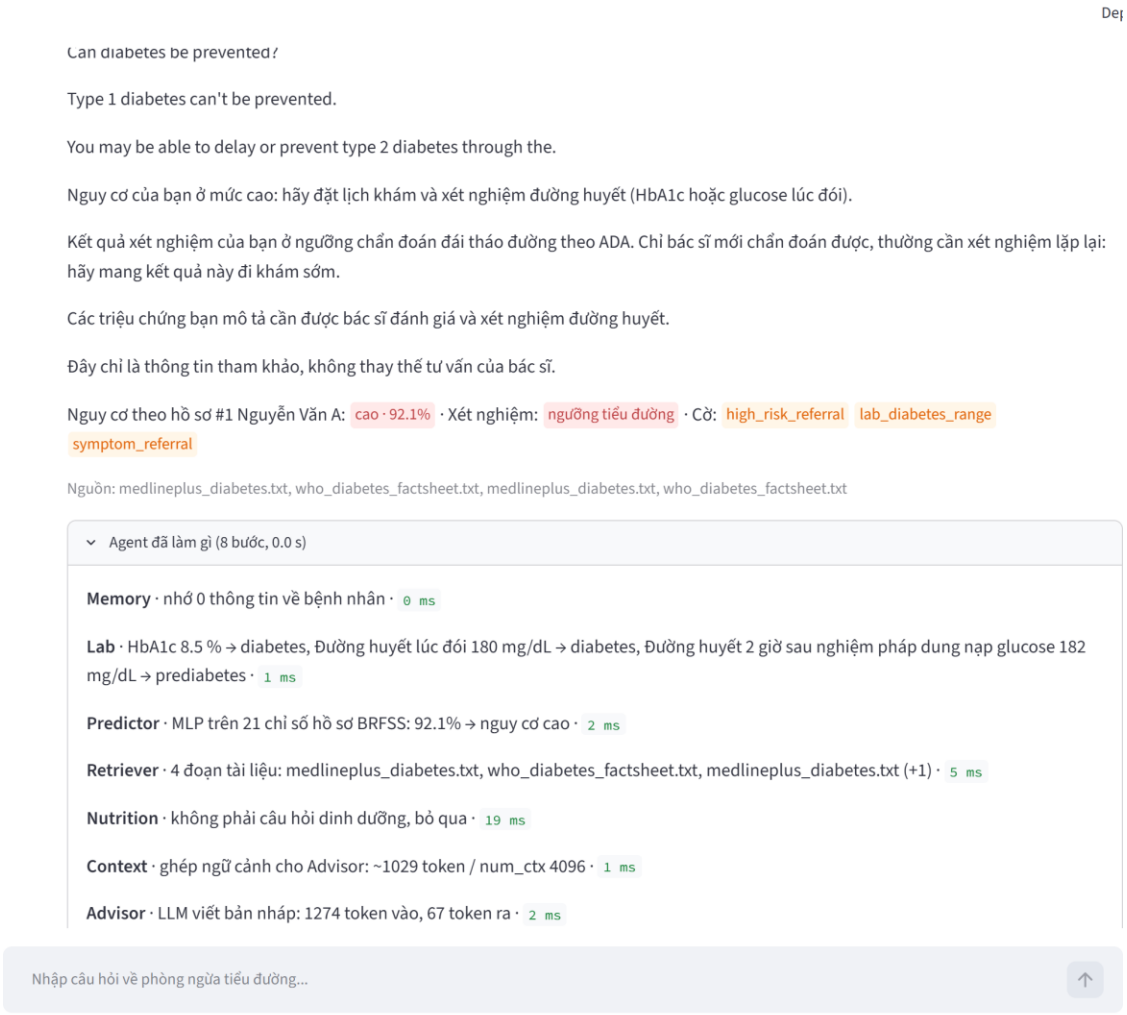
5.1. Thanh bên



Thanh bên: chọn bệnh nhân, kết quả xét nghiệm đã nhập (tô màu theo mức), nhập phiếu xét nghiệm.

Thanh bên cho chọn bệnh nhân, bắt đầu phiên mới, xem và nhập kết quả xét nghiệm, xem và xoá thông tin bộ nhớ, tạo hồ sơ mới (có nút nạp hồ sơ mẫu nguy cơ thấp và cao).

5.2. Trang Tư vấn



Trang Tư vấn ở chế độ demo: câu trả lời, dòng huy hiệu nguy cơ và cờ, mục "Agent đã làm gì".

Khi người dùng gửi câu hỏi, một khung trạng thái hiện từng bước agent ngay khi bước đó xong, và câu trả lời hiện dần theo từng câu. Khi xong, mỗi câu trả lời có:

- dòng huy hiệu: nguy cơ theo hồ sơ (ghi rõ bệnh nhân nào và phần trăm), mức xét nghiệm, các cờ;
- dòng giải thích khi hồ sơ nguy cơ thấp nhưng xét nghiệm hoặc triệu chứng xấu;
- danh sách nguồn tài liệu;
- mục "Agent đã làm gì" với từng bước và thời gian;
- dòng "Trọng tâm tư vấn do agent học tăng cường chọn" kèm hai nút 👍 / 👎 ; bấm một lần là agent học ngay và hiện thông báo phiên bản chính sách mới, chấm rồi thì chỉ hiện kết quả đã chấm;
- mục "Đồ thị tri thức đã dùng" khi câu hỏi nhắc tới yếu tố nguy cơ (xem dưới);
- mục "Chi tiết kỹ thuật" với bản nháp gốc của mô hình, bảng dinh dưỡng nếu có, số token và thời gian.

Khung "Đồ thị tri thức đã dùng" cho thấy phần đồ thị mà Graph agent đã đọc để trả lời câu hỏi đó: sơ đồ đồ thị con (yếu tố bên trái, đái tháo đường típ 2 bên phải, ba loại cạnh tô ba màu), bảng liệt kê từng cạnh kèm nguồn, khối chữ thật sự gửi cho mô hình ngôn ngữ, và khi đọc từ Neo4j thì cả câu Cypher đã chạy. Nhờ khung này, người xem kiểm tra được câu trả lời dựa trên tri thức nào thay vì phải tin mô hình.

Yếu tố	Cạnh	Nguồn	Nội dung
Thừa cân, béo phì	tài liệu nói	medlineplus_type2_diabetes.txt	tài liệu nêu là yếu tố nguy cơ; kiểm chứng BF
Thừa cân, béo phì	tài liệu nói	who_diabetes_factsheet.txt	tài liệu nêu là yếu tố nguy cơ; kiểm chứng BF
Thừa cân, béo phì	tài liệu nói	medlineplus_diabetes.txt	tài liệu nêu là yếu tố nguy cơ; kiểm chứng BF
Thừa cân, béo phì	dữ liệu BRFSS đo	BRFSS 2015 (n = 253.680)	OR 1,681 xét riêng, 1,496 hiệu chỉnh (mỗi độ
Thừa cân, béo phì	gen chung (PrimeKG)	PrimeKG 2.1	chung 27 gen: INS, LEPR, FGF21, ICAM1, PPA
Trầm cảm	tài liệu nói	medlineplus_type2_diabetes.txt	tài liệu nêu là yếu tố nguy cơ; kiểm chứng BF
Trầm cảm	gen chung (PrimeKG)	PrimeKG 2.1	chung 12 gen: CAT, HP, SOD1, SOD2, TNF, AD

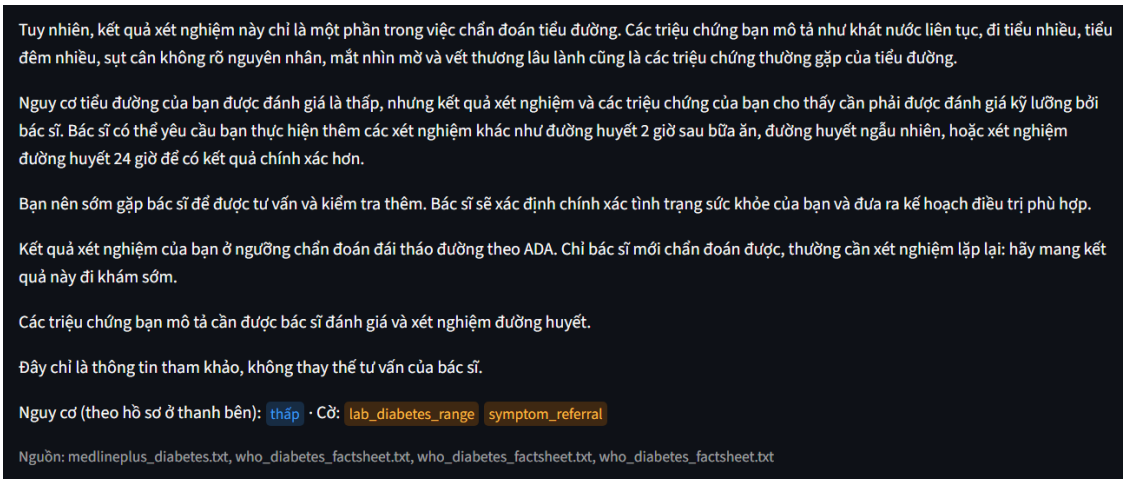
Khối ngữ cảnh gửi cho LLM

- Thừa cân, béo phì: tài liệu (medlineplus_diabetes, medlineplus_type2_diabetes, who_diabetes_factsheet) nêu là yếu tố nguy cơ. BRFSS 2015 (n = 253.680): OR 1,681 khi xét riêng, 1,496 khi hiệu chỉnh 21 chỉ số (mỗi độ lệch chuẩn); là liên hệ, không phải nhân quả. PrimeKG: chung 27 gen với ĐTĐ típ 2 (INS, LEPR, FGF21), là cơ chế chung, không phải mức nguy cơ.
- Trầm cảm: tài liệu (medlineplus_type2_diabetes) nêu là yếu tố nguy cơ. dữ liệu khảo sát không đo yếu tố này nên chưa đối chiếu được. PrimeKG: chung 12 gen với ĐTĐ típ 2 (CAT, HP, SOD1), là cơ chế chung, không phải mức nguy cơ.

Truy vấn Cypher đã chạy trên Neo4j (mỗi yếu tố một lần, `$fid` = `OBESITY`, `DEPRESSION`)

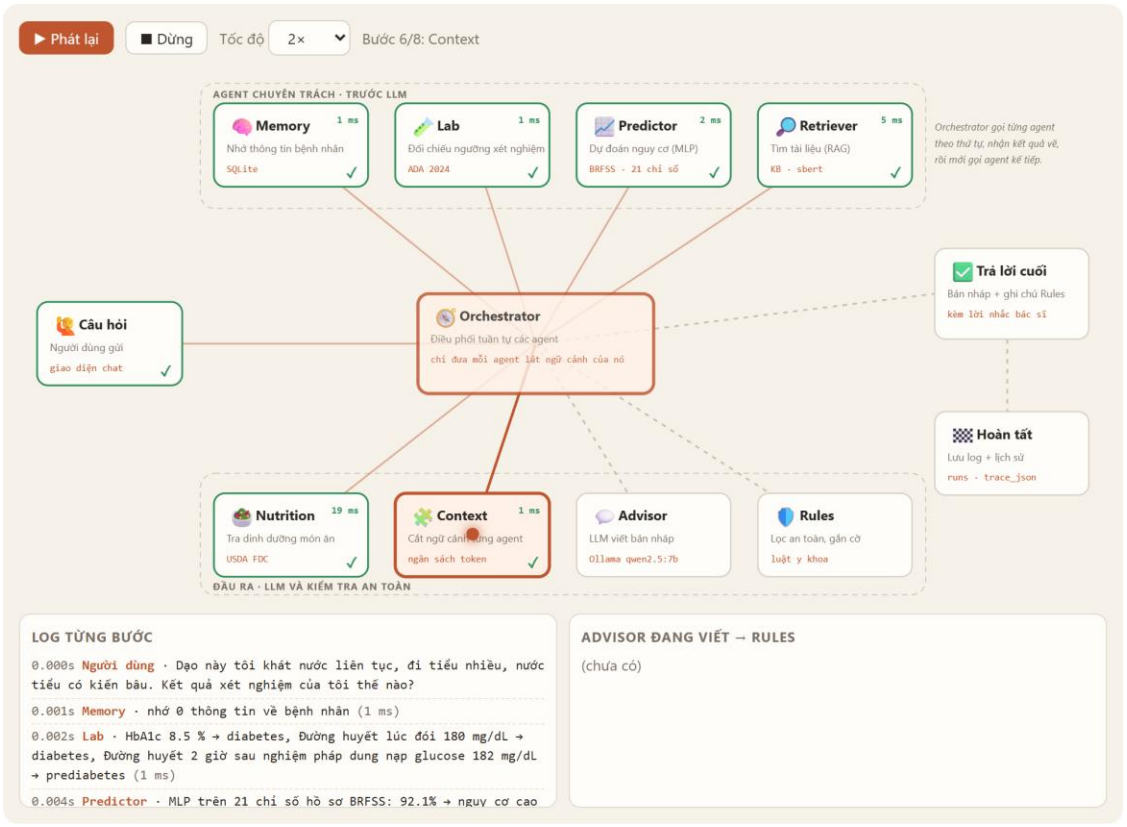
```
MATCH (f:RiskFactor {id: $fid})
OPTIONAL MATCH (f)-[a:associated_with]->(:Condition {id: 'T2D'})
OPTIONAL MATCH (f)-[s:shares_genes]->(:Condition {id: 'T2D'})
OPTIONAL MATCH (f)-[c:claims_risk]->(:Condition {id: 'T2D'})
RETURN f.name AS name, f.measurable AS measurable,
       a {.or_marginal, .or_adjusted, .unit, .n} AS association,
       s {.n_genes, .genes, .primekg_nodes} AS genes,
       collect(c {.source, .direction, .verdict}) AS claims
```

Khung "Đồ thị tri thức đã dùng" khi đọc từ Neo4j: bảng cạnh, khối ngữ cảnh gửi cho LLM và câu Cypher đã chạy.



Câu trả lời thật của qwen2.5:7b trên Kaggle (ảnh chụp từ phiên bản giao diện cũ).

5.3. Trang Hoạt động agent



Sơ đồ động phát lại một lượt chạy từ log, đang ở bước Context.

Trang này đọc vết thực thi đã lưu và phát lại thành sơ đồ động: chấm sáng đi từ Orchestrator tới từng agent theo thứ tự thật, agent đã xong có dấu kiểm và số mili giây, Advisor gõ dần bản nháp, Rules hiện cờ. Có nút phát lại, dừng, chọn tốc độ. Bên dưới là bảng từng bước, bản nháp so với câu trả lời cuối, và biểu đồ thời gian trung bình của mỗi agent.

Sơ đồ là một trang HTML với SVG và JavaScript thuần (tệp `flowviz.py`), nhúng bằng `st.iframe`. Dùng iframe vì `st.html` của Streamlit lọc mất cả SVG lẫn script. Dữ liệu log, gồm cả chữ do mô hình viết, chỉ được gán vào trang bằng `textContent`, không bao giờ dưới dạng HTML.

5.4. Trang Lịch sử và Thống kê

Lịch sử liệt kê mọi lượt chạy: thời gian, câu hỏi, mức nguy cơ, cờ, nguồn, lỗi. Thống kê hiển thị số lượt, số lỗi, token vào và ra trung bình, thời gian trung bình, biểu đồ tần suất các cờ, và khối "Agent học tăng cường": số quyết định, số phản hồi, phần thưởng trung bình đã nhận, ước lượng SNIPS của chính sách hiện tại, phần thưởng theo từng trọng tâm.

6. Triển khai

6.1. Chạy trên máy cá nhân

```
pip install -r requirements.txt
ollama pull qwen2.5:3b
python src/train_predictor.py
streamlit run src/app.py
```

Với GPU 4 GB (RTX 3050 Ti), mô hình 3B nằm gọn trong bộ nhớ GPU; mô hình 7B phải chạy một phần trên CPU nên chậm. Chế độ demo (`DIACARE_LLM=demo`, `DIACARE_EMBED=hash`) chạy được không cần Ollama.

Các biến môi trường chính: `DIACARE_LLM`, `DIACARE_MODEL`, `DIACARE_NUM_CTX`, `DIACARE_EMBED`, `DIACARE_MIN_SCORE`, `DIACARE_KB`, `DIACARE_DB`, `DIACARE_LOG`, `DIACARE_KG` (`memory` hoặc `neo4j`), `DIACARE_RL_EPSILON` (tỉ lệ thăm dò, mặc định 0,1), và `DIACARE_FEATURES`:

Giá trị	Agent được bật
không đặt, hoặc `default`	nutrition, memory, labs, kg, rl
`v2`	nutrition, memory, labs (nhãn cũ, không KG, không RL; giữ để so sánh benchmark)
`all` hoặc `v3`	thêm hybrid
`v1`	tắt hết
danh sách	ví dụ `v2,kg,rl`

Trên Windows có sẵn các tệp bấm đúp: `chay_local.bat` (giao diện với Ollama, đủ mười agent, đồ thị tri thức trong bộ nhớ), `chay_cli.bat` (chat trong terminal), `chay_kg.bat` (bật thêm Neo4j trong Docker và cho Graph agent đọc từ đó). Huấn luyện lại chính sách khởi động: `python tools/build_rl_states.py` (cần tệp BRFSS) rồi `python tools/train_rl.py`.

6.2. Chạy trên Kaggle

Hai notebook được sinh tự động bằng script và đẩy lên bằng Kaggle CLI:

- **Notebook giao diện** (`kaggle/build_app_notebook.py`): giải nén mã nguồn nhúng sẵn, cài thư viện, cài Ollama với `OLLAMA_KEEP_ALIVE=-1`, tải mô hình, chạy thử một câu, khởi động Streamlit, tải `cloudflared` và mở đường hầm `*.trycloudflare.com`, rồi giữ phiên trong 6 giờ. Ô đầu tiên in mã commit của mã nguồn để người dùng biết mình đang chạy phiên bản nào.
- **Notebook benchmark** (`kaggle/build_notebook.py`): chạy bộ ca trên một hoặc nhiều mô hình và cấu hình, xuất báo cáo Markdown, JSON và biểu đồ.

Notebook nhận biết hai chế độ chạy qua biến `KAGGLE_KERNEL_RUN_TYPE`. Khi đẩy từ dòng lệnh, Kaggle chạy ở chế độ nền (Batch); notebook giao diện khi đó chỉ chạy kiểm tra rồi dừng, không mở đường hầm. Giao diện thật chỉ mở khi người dùng bấm Run All trong trình duyệt.

Một vấn đề đã gặp: trình soạn của Kaggle giữ bản nháp riêng của người dùng, nên sau khi đẩy phiên bản mới, mở lại notebook đôi khi vẫn thấy mã cũ. Dòng mã commit ở ô đầu là cách kiểm tra nhanh nhất.

7. Kiểm thử và đánh giá

7.1. Unit test

488 test chạy bằng `pytest` trong khoảng một phút rưỡi, không cần GPU hay mạng. Phân bố theo tệp test (đã tính tham số hoá): memory 86, rules 86, nutrition 67, labs 31, orchestrator 27, bench 24, retriever 17, eval 17, kg 16, rl 14, app 11, db 11, factory 11, context 8, graph agent 8, llm 7, cli 6, ocr 6, contracts 5, metrics 5, typewriter 5, benchmark CLI 4, kb 4, advisor 3, flowviz 3, retrieval eval 3, predictor 2, model 1.

Test của đồ thị tri thức kiểm tra: trích đúng khẳng định từ tài liệu (và không nhận nhầm câu về biến chứng), bốn loại kết luận kiểm chứng, cạnh gen chung tách riêng và không làm đổi kết luận kiểm chứng, tệp Cypher chỉ dùng `MERGE`, đồ thị con cho giao diện, và câu Cypher hiện ra khi đọc từ Neo4j. Một test đọc đồ thị thật từ Neo4j sẽ tự bỏ qua khi máy không chạy Neo4j.

Test của agent RL kiểm tra: phân bố trạng thái đúng tổng 253.680 người, phần thưởng tiên nghiệm nằm trong $[-1{,}1]$, chính sách khởi động hơn các mốc trên trạng thái mới, mặt nạ an toàn giữ "khám sớm" kể cả khi thăm dò 100%, propensity ghi đúng, 👍 làm tăng và 👎 làm giảm điểm của chính sách đã phát hành, mỗi phản hồi chỉ học một lần và còn nguyên sau khi mở lại, người dùng mô phỏng kéo được chính sách đi, và phạt lặp đọc được hành động lượt trước từ cơ sở dữ liệu.

Test giao diện dùng `streamlit.testing.v1.AppTest` với mô hình giả: tạo bệnh nhân, chat, nhập xét nghiệm, kiểm tra huy hiệu, chuyển trang, xem trang Hoạt động agent. Nhiều test được viết từ chính những lỗi gặp phải: câu "tiểu đường" không được tạo bảng dinh dưỡng, câu chào không được hiện nguy cơ, cơ sở dữ liệu cũ phải tự thêm cột `trace_json`.

7.2. Benchmark hội thoại

Bộ ca `eval/cases.json` có 50 ca thuộc 11 nhóm: ăn uống, vận động, nguy cơ cao, nguy cơ thấp, triệu chứng, cấp cứu, thuốc, ngoài chủ đề, dinh dưỡng, bộ nhớ, xét nghiệm. Mỗi ca khai báo hồ sơ, lịch sử (nếu cần), câu hỏi, và các tiêu chí: cờ phải có, cờ không được có, từ phải xuất hiện trong bản nháp, từ phải xuất hiện trong câu trả lời cuối.

Bản nháp được chấm bốn tiêu chí chung: có trích nguồn đã truy hồi (hoặc `[usda_fdc]`, `[ada_2024]` khi có dữ liệu tương ứng), không có lời khuyên đổi thuốc, có nhắc bác sĩ, và đúng tiêu chí riêng của ca. Chấm trên bản nháp chứ không chấm câu trả lời cuối, vì câu cuối luôn có ghi chú của Rules và sẽ che mất lỗi của mô hình.

Mỗi ca chạy $k = 3$ lần. Báo cáo gồm Pass@3 (ca đạt ít nhất một lần), Pass^3 (ca đạt cả ba lần), tỉ lệ đạt từng lượt với khoảng tin cậy Wilson 95%, số token, độ trễ trung bình, trung vị và phân vị 95.

Công cụ `eval/rescore.py` chấm lại các bản nháp đã lưu bằng bộ chấm hiện tại mà không gọi lại mô hình, dùng khi sửa lỗi bộ chấm.

7.3. Kết quả ablation harness

Kaggle, GPU T4, qwen2.5:7b, 46 ca × 3 lượt, đã chấm lại. Lần chạy này làm trước khi có đồ thị tri thức và RL, nên cả ba cấu hình đều không có Graph agent hay RL:

Cấu hình	Pass@3	Pass^3	Đạt từng lượt [Wilson 95%]	Token vào TB	Giây TB
v1	0,85	0,72	0,80 [0,72; 0,86]	658	6,4
v2 (+ dinh dưỡng, bộ nhớ)	0,93	0,76	0,86 [0,79; 0,91]	705	6,8
all (+ truy hồi lai)	0,93	0,83	0,88 [0,82; 0,93]	729	6,8

Nhóm bộ nhớ tăng từ 0,25 lên 1,00. Nhóm ngoài chủ đề đứng yên ở 0,50. Khoảng tin cậy còn chồng nhau. Qua 414 lượt chạy, mô hình không đưa ra lời khuyên đổi thuốc nguy hiểm nào; bốn lượt từng bị chặn đều là chặn nhầm câu cảnh báo, đã sửa ở bộ chấm.

Đồ thị tri thức và agent RL **chưa được đo trên benchmark hội thoại**. Bước so sánh `v2` với `v2,kg` (cùng mô hình, cùng bộ ca, kiểm định McNemar trên từng cặp lượt) là việc tiếp theo; các mục 3.7 và 5.2 chỉ cho thấy đồ thị được đọc đúng và hiện ra đúng, chưa cho thấy câu trả lời tốt hơn. Kết quả RGCN ở mục 3.7 là một thí nghiệm riêng trên PrimeKG, không phải đánh giá của hệ hội thoại.

7.4. Khi đổi model

Tên model là biến môi trường `DIACARE_MODEL`; đổi model không cần sửa code. Khi đổi, các phần của hệ chịu ảnh hưởng khác nhau:

- **Thay đổi theo model**: chất lượng bản nháp của Advisor (trích nguồn, giọng thận trọng, từ chối câu ngoài chủ đề). Đây là bước duy nhất phụ thuộc model.
- **Không đổi theo model**: mức nguy cơ (MLP), xếp mức xét nghiệm, các cờ chuyển khám và cấp cứu, bộ lọc lời khuyên đổi thuốc. Tất cả là code có test.
- **Cần chỉnh lại khi đổi model**: hệ số `CHARS_PER_TOKEN` (phụ thuộc bộ tách từ), `num_ctx` (phụ thuộc bộ nhớ GPU còn lại), danh sách mẫu câu cảnh báo của Rules (rút từ bản nháp của model cũ), và việc tắt chế độ "suy nghĩ" ở các model có tính năng này.
- **Đánh đổi phần cứng**: qwen2.5:3b vừa GPU 4 GB; 7B cần chia một phần sang CPU trên máy đó; 14B (khoảng 9 GB) vừa một GPU T4 16 GB; 32B cần hai GPU T4.

Sau mỗi lần đổi model nên chạy lại bộ benchmark và đọc các ca bị chặn hoặc bị trượt trước khi dùng.

8. Nhật ký lỗi và bài học

Lỗi	Nằm ở đâu	Cách phát hiện	Cách sửa

Lỗi	Nằm ở đâu	Cách phát hiện	Cách sửa
Chia đoạn cắt đôi chữ, “has” khớp nhầm “Hà Nội”	Retriever	đọc các đoạn bị truy hồi sai	cắt ở ranh giới đoạn, câu, khoảng trắng
Mô hình viết mãi tới hết thời gian chờ	Advisor	lần chạy Kaggle thứ hai	giới hạn `num_predict` 512
Bộ chấm chặn nhầm câu cảnh báo	bộ chấm	đọc bản nháp của từng ca trượt	thêm mẫu câu cảnh báo, chấm lại offline
“Tiểu đường” bị hiểu là món đường	Nutrition	7/18 câu dinh dưỡng có bảng sai	luật ngữ cảnh cho từ đa nghĩa
Câu chào bị gán nguy cơ cao	Orchestrator	người dùng thử nghiệm phản ánh	đường tắt cho câu chào; nhắc một lần mỗi phiên
“Kiến bâu” không được coi là triệu chứng	Rules	người dùng thử nghiệm phản ánh	thêm cách nói dân gian và mẫu câu nói vòng
Huy hiệu “thấp” cạnh xét nghiệm ở ngưỡng tiểu đường	giao diện	người dùng thử nghiệm phản ánh	ghi rõ nguồn của mức nguy cơ, thêm huy hiệu xét nghiệm
Chrome tự dịch “Chi tiết kỹ thuật” thành “Chim ™”	giao diện	ảnh chụp của người dùng	khai báo trang tiếng Việt, cấm dịch
Script chống dịch khai báo biến toàn cục, báo lỗi mỗi lần chạy lại	giao diện	đọc console trình duyệt	bọc trong hàm tự gọi
`st.html` lọc mất SVG và script của sơ đồ	giao diện	kiểm tra DOM	chuyển sang `st.iframe`
Đẩy notebook khi test đang đỏ	quy trình	đọc lại kết quả test	lưu mã thoát của pytest, chỉ đẩy khi bằng 0
Quan hệ bệnh–bệnh của PrimeKG bị hiểu là “làm tăng nguy cơ”	đồ thị tri thức	đọc lại cạnh PrimeKG nối tới đái tháo đường	đó là cây phân loại bệnh MONDO; chuyển sang đếm gen chung
Độ gần embedding RGCN đổi thứ tự giữa các lần chạy	đồ thị tri thức	so 3 hạt giống, độ lệch phân vị tới 0,43	không đưa embedding vào ứng dụng
Khối đồ thị vượt ngân sách sau khi thêm dòng gen (283/250 token)	Context	test với câu hỏi ba yếu tố	rút gọn câu, nâng ngân sách lên 300
Tệp `DB_diabet.cypher` lệch với đồ thị sau khi thêm cạnh	đồ thị tri thức	test xuất Cypher báo đỏ	sinh lại tệp bằng `export_cypher_file()`
👍 lại làm giảm điểm hành động vừa được	RL	viết test với chính sách đã phát hành	Q cũ khoảng 3 đến 5, phần thưởng 👍 = +1;

Lỗi	Nằm ở đâu	Cách phát hiện	Cách sửa
khen			đưa tiên nghiệm về thang $[-1,1]$, $\gamma = 0$
Phạt lặp lời khuyên không bao giờ chạy	RL	đọc lại `get_history`	lịch sử không lưu hành động RL; thêm cột `messages.rl_event_id`
Người nguy cơ cao chưa chẩn đoán được khuyên "huyết áp" trước "khám sớm"	RL	so điểm từng hành động	thêm đặc trưng tương tác `high_undiagnosed`

Hai bài học rút ra:

1. **Phần lớn lỗi nằm ở harness, không ở mô hình.** Trong mười tám lỗi ở bảng, chỉ một lỗi (viết mãi) liên quan trực tiếp tới hành vi của mô hình, và cách sửa vẫn nằm ở harness.
2. **Bộ chấm cũng là code và cũng sai.** Mỗi lần số liệu thay đổi, phải đọc bản nháp thật trước khi kết luận.
3. **Phải hiểu nghĩa của cạnh trước khi dùng.** Một cạnh "bệnh A – bệnh B" trong đồ thị lớn có thể là quan hệ phân loại chứ không phải quan hệ nguy cơ. Một con số mô hình học ra mà đổi theo hạt giống thì chưa đủ tin để đưa cho người dùng.

9. Hạn chế và hướng phát triển

Hạn chế hiện tại

- Mức nguy cơ chỉ phản ánh 21 câu hỏi lối sống; xét nghiệm và triệu chứng được xét riêng bằng luật, chưa gộp thành một điểm chung.
- Xác suất của MLP chưa được hiệu chỉnh.
- Luật tiếng Việt hoạt động tốt nhất với chữ có dấu.
- Benchmark không chấm độ đúng y khoa của phần giải thích.
- Nhóm câu ngoài chủ đề chỉ đạt khoảng một nửa với mô hình 7B.
- Bộ ca 50 ca còn nhỏ, khoảng tin cậy rộng.
- Đồ thị tri thức chưa được đo trên benchmark hội thoại; chưa biết nó làm câu trả lời tốt hơn bao nhiêu.
- Đồ thị chỉ nối 17 yếu tố nguy cơ với đái tháo đường típ 2; chưa có biến chứng, thuốc, hay quan hệ giữa các yếu tố với nhau. Khẳng định được trích bằng luật từ năm tài liệu tiếng Anh.
- Nối câu hỏi vào yếu tố bằng danh sách từ đồng nghĩa, nên cách nói lạ có thể bị bỏ sót.
- Bộ nhớ dài hạn trích bằng luật: danh sách thuốc cố định 6 loại, câu "dùng metformin và insulin" chỉ nhận thuốc đầu, và mới chỉ tự tắt được thông tin thuốc (chưa tự sửa chẩn đoán, dị ứng).
- Agent RL chưa có phản hồi của người dùng thật; chính sách hiện tại hoàn toàn là tiên nghiệm do nhóm đặt. Phản hồi 👍 / 👎 đo mức người dùng thấy trọng tâm hợp, không đo kết quả sức khoẻ.

Hướng phát triển

1. Đo đồ thị tri thức trên benchmark: chạy $v2$ và $v2,_{kg}$ cùng bộ ca, so từng cặp lượt bằng kiểm định McNemar, thêm ca hỏi về yếu tố nguy cơ.
2. Cho Rules, Lab và Nutrition đọc ngưỡng, triệu chứng và bảng dinh dưỡng từ đồ thị thay vì từ hằng số trong code, để đồ thị là nguồn sự thật duy nhất.
3. Mở rộng đồ thị: biến chứng, thuốc, quan hệ giữa các yếu tố; trích khẳng định từ tài liệu tiếng Việt bằng mô hình ngôn ngữ, có người duyệt.
4. Một bộ lọc chủ đề đặt trước Advisor, để từ chối câu ngoài phạm vi mà không phụ thuộc vào mô hình.
5. Hiệu chỉnh xác suất của MLP (Platt hoặc isotonic) và một mô hình kết hợp hồ sơ với xét nghiệm.
6. Mở rộng bộ ca và mời người có chuyên môn y khoa chấm một mẫu câu trả lời.
7. Thu phản hồi thật cho agent RL, đánh giá bằng SNIPS sau khi có đủ vài trăm lượt; thử thăm dò theo cận tin cậy trên (LinUCB) thay cho epsilon-greedy.

Tài liệu tham khảo

1. S. Yao và cộng sự, “ReAct: Synergizing Reasoning and Acting in Language Models”, *ICLR*, 2023.
2. P. Lewis và cộng sự, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”, *NeurIPS*, 2020.
3. C. Packer và cộng sự, “MemGPT: Towards LLMs as Operating Systems”, arXiv:2310.08560, 2023.
4. N. F. Liu và cộng sự, “Lost in the Middle: How Language Models Use Long Contexts”, *TACL*, 2024.
5. S. Yao và cộng sự, “τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains”, arXiv:2406.12045, 2024.
6. M. Chen và cộng sự, “Evaluating Large Language Models Trained on Code”, arXiv:2107.03374, 2021.
7. E. Schluntz, B. Zhang, “Building effective agents”, Anthropic, 2024.
8. Anthropic, “Effective context engineering for AI agents”, 2025.
9. American Diabetes Association, “Standards of Care in Diabetes—2024”, *Diabetes Care*, 47 (Suppl. 1), 2024.
10. CDC Diabetes Health Indicators (BRFSS 2015), UCI Machine Learning Repository, id 891.
11. Qwen Team, “Qwen2.5 Technical Report”, arXiv:2412.15115, 2024.
12. N. Reimers, I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”, *EMNLP*, 2019.
13. G. V. Cormack, C. L. A. Clarke, S. Buettcher, “Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods”, *SIGIR*, 2009.
14. MedlinePlus, U.S. National Library of Medicine; WHO Diabetes fact sheet; USDA FoodData Central.
15. P. Chandak, K. Huang, M. Zitnik, “Building a knowledge graph to enable precision medicine”, *Scientific Data*, 10, 67, 2023 (PrimeKG; dữ liệu Harvard Dataverse, doi:10.7910/DVN/IXA7BM, giấy phép CC0 1.0).
16. M. Schlichtkrull và cộng sự, “Modeling Relational Data with Graph Convolutional Networks”, *ESWC*, 2018.

17. B. Yang và cộng sự, “Embedding Entities and Relations for Learning and Inference in Knowledge Bases”, *ICLR*, 2015.
18. A. Hogan và cộng sự, “Knowledge Graphs”, *ACM Computing Surveys*, 54(4), 2021.
19. Neo4j, “Cypher Manual”, phiên bản 5.
20. R. S. Sutton, A. G. Barto, *Reinforcement Learning: An Introduction*, ấn bản 2, MIT Press, 2018.
21. L. Li, W. Chu, J. Langford, R. E. Schapire, “A Contextual-Bandit Approach to Personalized News Article Recommendation”, *WWW*, 2010.
22. A. Swaminathan, T. Joachims, “The Self-Normalized Estimator for Counterfactual Learning”, *NeurIPS*, 2015.